

实验一报告：Pure Fixed Budget 下自适应 Bundle 方法的 λ 求解器对比 (CCP vs IPOPT)

$K = 2$ 与 $K = 6$ 两个实例；2026 年 8 月 9 日同机串行重跑 (方案 A)。

0. 实验协议概述 (两个实例共用)

实验采用 pure fixed budget 协议：所有方法花费完全相同的梯度当量预算 B ，共享同一个执行循环——工作单元为一个 segment (epoch_len 步 Momentum-SVRG 小批量更新 + 一次全量 joint 评估)，每次“决策”连续花费 s 个 segment，链式 warm start，预算花完即停，全程无任何容差参数。各方法唯一的差别是“下一个 λ 选在哪里”的策略；决策耗时计入该方法自己的 CPU 轴。

公平性设计有三条：(1) 每个方法使用自己完整的 targeting 机制 (不共享种子——起点策略本身是被比较对象的一部分)；(2) 质量永远由同一把 off-axis 的统一 audit 尺子事后测量 (不计入任何成本轴)；(3) 目标归一化固定 $s_k \equiv 1$ ，两臂共享同一 problem 实例 (相同数据种子、初始点与 L 校准)， λ 求解层不存在任何按方法或按 bundle 的再缩放。

问题族：planted linear-softmax 数据 + [96, 96] tanh MLP 的 per-class 交叉熵目标 (每个类别一个目标 F_k)， $n = 50000$ 样本、 $p = 20$ 维特征；两个实例 $K = 2$ 与 $K = 6$ 。

1. $K = 2$ 实例 ($B = 20000$)

1.1 参与对比的算法

腿 (leg)	下一个 λ 的策略	说明
baseline_r{10,20,40,80}_s5 (4 条)	蛇形排序的均匀网格轮转	分辨率 r 决定网格密度 ($r=10 \rightarrow 11$ 个节点)；决策免费；纯扫描式基线
adaptive_s5_ts24 (IPOPT)	对当前 Gram 栈做 strict 多起点 IPOPT 搜索	结构化起点集在 $K=2$ 实际为 7 个 (质心/顶点 $\times 2$ /近角点 $\times 2$ /上一轮 λ /边中点)；ts=24 只是上限帽；单点 prev_lam 链式热启动
adaptive_s5_ccp (CCP)	多起点凸凹过程 (gns-ccp 论文 Algorithm 1)	种子 = 顶点 + λ_A + 跨轮候选池 + $N_0=2000$ 个 Exp(1) 随机点 $\rightarrow$ 批量筛选取前 $r=10 \rightarrow$ 热启动 HiGHS 博弈 LP 逐个精修；池上限 $3r=30$

1.2 实验参数

参数	作用	数值	备注
B (budget)	总梯度当量预算，唯一停机规则	20000	两臂逐条相同
s	每次决策连续花费的 segment 数	5	所有腿共享
eval_every	checkpoint 记录间隔 (梯度当量)	250	只影响记录，不影响算法
K / p / n	类别数 / 特征维数 / 样本数	2 / 20 / 50000	planted 合成数据，data_seed=7
hidden_sizes / 激活	MLP 结构	[96, 96] / tanh	d ≈ 1.19 万参数；init_seed=8
msvrg_batch / epoch_len	小批量大小 / 每 segment 步数	4096 / 13	epoch_len = $\lceil n/\text{batch} \rceil$ ；sampler_seed=41
step_const / momentum	内层步长常数与动量	0.1 / 0.5	步长 = $\text{step_const}/(\lambda^T L \cdot L_scale)$ ，安全阀倍增 L_scale
targeting_starts (IPOPT 腿)	多起点搜索的起点数上限	24	K=2 结构化全集只有 7 个，上限不生效
N0 / r (CCP 腿)	随机种子数 / 精修起点数	2000 / 10	采样器 Exp(1)；自适应种子调度关闭
pool_cap (CCP 腿)	跨轮候选池上限	3r = 30	后期 distinct=31，每轮截 1 个 (已记录 4r 消融候选)
τ (CCP 停机)	CCP 内层平稳残差容差	$1e-8 \cdot \max(1, \varphi)$	pure budget 无 ϵ ，相对项独立生效
audit_grid	精确 audit 的网格密度	200001	仅测量用，off-axis

1.3 统一 audit (质量尺子)

K = 2 拥有精确尺子： $\lambda = (w, 1-w)$ 参数化下每个 $\varphi_i(w)$ 是显式凸二次函数，交付集的真实 $GN^* = \max_w \min_i \varphi_i(w)$ 通过 20 万点稠密网格 + 获胜邻域的抛物线交点精确求解得到，并附带认证上界 (本次全程认证间隙 $\leq 6.5e-7$)。该 audit 在每条腿的每个 checkpoint 上运行，时间不计入任何成本轴。所有质量数字与曲线均出自这把尺子，与各方法自己的搜索无关。

1.4 结果图与分析

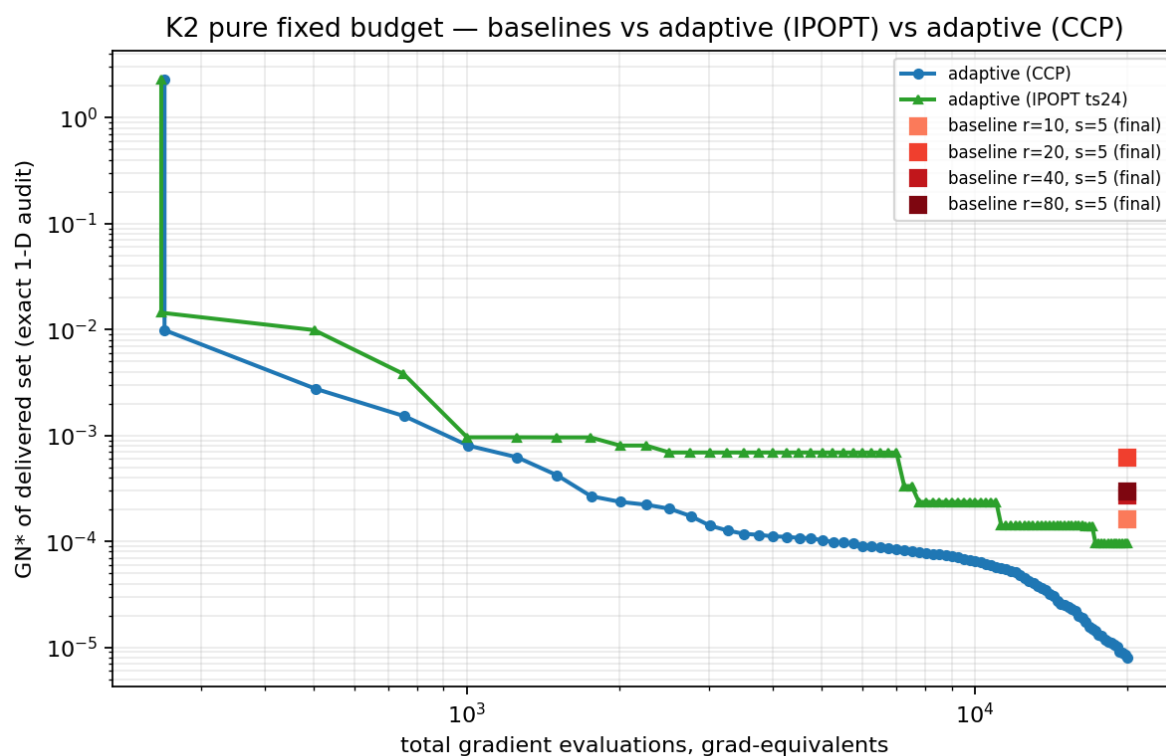


图 1-1 K2 : 交付集 GN^* (精确 audit) 随梯度当量预算的变化

分析：横轴是花掉的梯度当量（两条 adaptive 腿的开销逐条相同），纵轴是每个 checkpoint 交付集的精确 GN^* 。终点：CCP $7.93e-6$ ，IPOPT $9.52e-5$ ，最好的 baseline（ $r=10$ ） $1.61e-4$ 。两条 adaptive 腿都远优于全部 baseline，说明“把预算花在当前最差方向”的自适应策略本身成立；而在完全相同的梯度花费下 CCP 又比 IPOPT 低一个数量级——这个差距只可能来自“每次决策选的 λ 更准”，与花费无关。

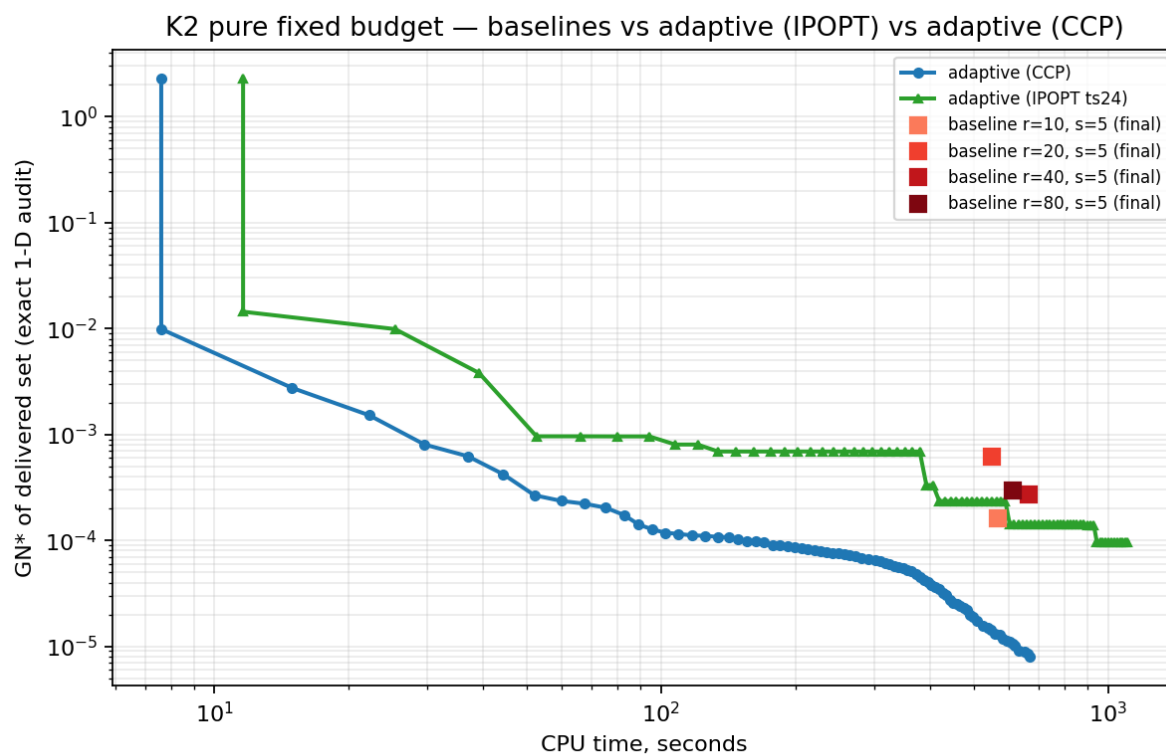


图 1-2 K2 : 交付集 GN^* 随 CPU 时间的变化

分析：换到 CPU 轴后差距进一步拉大：IPOPT 腿的 targeting 花 560.1 s (占整腿 1243.6 s 的 45%)，CCP 腿只花 103.5 s (占 802.7 s 的 13%)。CCP 用约一半的 CPU 时间达到并越过 IPOPT 的最终质量。基线各腿以最终点标注 (决策免费，其 CPU 都花在 segment 上)。



图 1-3 K2 : 每次决策的 targeting 最优性 gap (精确尺子)

分析：这张图度量“每次决策选的 λ 距离当时真正最差方向差多少”，即 $gap = \varphi_{exact} - \varphi(\lambda_{chosen})$ ，由精确尺子逐决策计算。CCP 的 gap 稳定在 $1e-9$ 量级（数值意义上每次都解到了精确解）；IPOPT 长期停在 $1e-3 \sim 1e-5$ 的平台——7 个结构化起点系统性错过全局最差方向。这是图 1-1 中质量差 12 倍的机制性解释：IPOPT 不是内层没解好（它每个局部解都收敛），而是多起点覆盖不足。

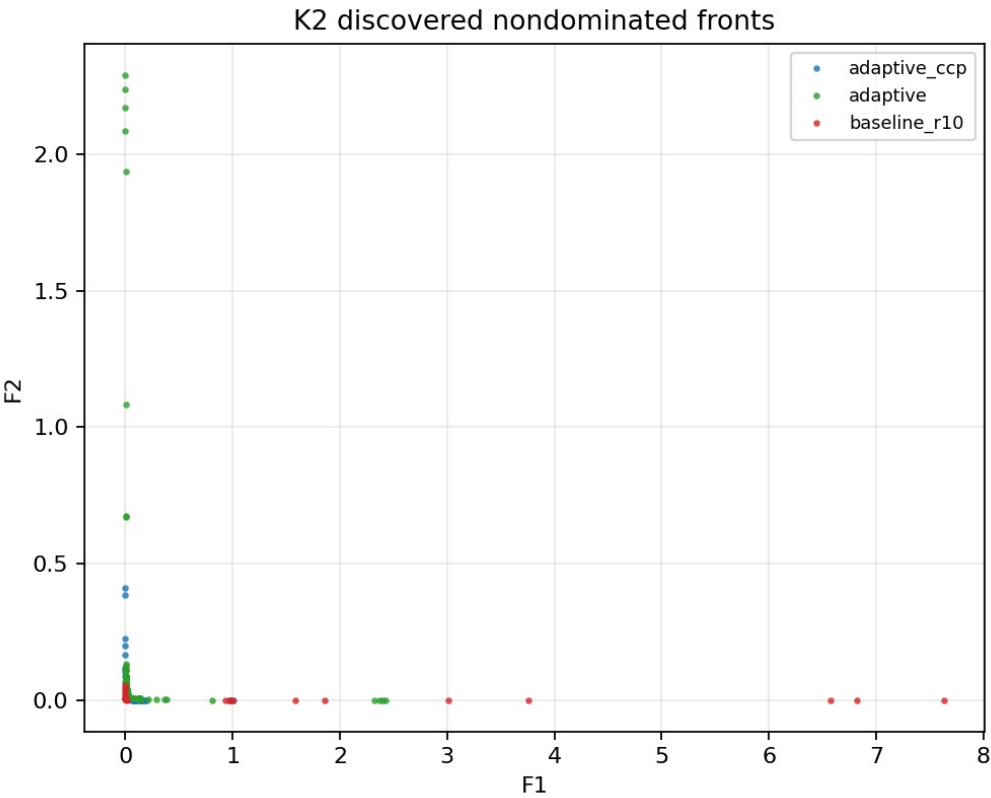


图 1-4 K2：各方法发现的非支配前沿（F1, F2 平面）

分析：前沿发现是 pure budget 协议的副产品指标。对合并前沿的 IGD：baseline_r10 = 0.087（35 点）、IPOPT = 0.365（88 点）、CCP = 0.553（81 点）。基线在前沿均匀度上最好是预期行为——它的本职就是均匀扫 λ 网格；两条 adaptive 腿把预算集中在最差方向附近，前沿点更密集于局部。CCP 与 IPOPT 的前沿点数和覆盖水平相近，说明更换 λ 求解器不损害前沿发现能力。

1.5 对应的 py 文件与输出

文件	角色
run_pure_budget_K2_without_256_checkpoints.py	协议执行器 + 精确 1-D audit + baseline/IPOPT 两种策略（7 月原文件，未改动）
run_pure_budget_K2_ccp_without_256_checkpoints.py	CCP 腿（新增；策略换为 CCPLambdaSolver，其余全部复用）

run_ccp_compare_K2_without_256_checkpoints.py	六条腿同机串行重跑的编排器 (新增)
ccp_lambda_solver.py	多起点 CCP λ 求解器 (核心新模块)
algorithm_fast_without_256_checkpoints.py	IPOPT 多起点搜索 _maximise_GN_fast (被 targeting 调用)
plot_ccp_compare_without_256_checkpoints.py	本节全部图 (新增)
输出目录 : output/ ccp_compare_without_256_checkpoints/K2_B20000/	六条腿 summary/grams + 图 + campaign_manifest.json

2. K = 6 实例 (B = 80912)

2.1 参与对比的算法

腿 (leg)	下一个 λ 的策略	说明
baseline_r{10,15}_s1、 baseline_r{10,12,15,20}_s5 (6 条)	蛇形排序的均匀网格轮转	K=6 时 r=10 的网格有 3003 个节点，预算内扫不完一遍；s=1 与 s=5 两种停留长度
adaptive_s5_ts24 (IPOPT)	strict 多起点 IPOPT 搜索	K=6 结构化全集 29 个起点，被 ts=24 截断；其余同 K2
adaptive_s5_ccp (CCP)	多起点凸凹过程	配置与 K2 完全一致 (N0=2000、r=10、池 3r)

2.2 实验参数 (仅列与 K2 不同处)

参数	作用	数值	备注
B (budget)	总梯度当量预算	80912	
eval_every	checkpoint 间隔	2000	约 41 个 checkpoint
K	类别数	6	问题族其余设定与 K2 相同
baseline 腿	网格分辨率×停留	r∈{10,12,15,20}, s∈{1,5}	共 6 条
targeting_starts	IPOPT 起点上限	24	结构化全集 29 → 实际 24 个
统一 audit	质量尺子	audit_v2 (双工具取 max)	见 2.3；strict-64 + CCP(N0=8192, r=20)

2.3 统一 audit : audit_v2 (双工具取 max)

K = 6 没有精确尺子 (认证代价随 K 指数增长)，历史上使用 strict 64 起点 IPOPT 搜索作为 audit。本次升级为方法对称的双工具尺子：audit_v2(栈) = max(strict-64 的值, 重配置 CCP

($N_0=8192$ 、 $r=20$ 、单轮无池) 的值)。两个工具都是真实 GN^* 的下界，取最大值是更紧的下界，且尺子对两个方法对称。

升级被数据证明是必要的：在 IPOPT 腿自己的交付栈上，CCP 工具在 42/42 个 checkpoint 上全部找到更高的真值（最终栈：strict-64 报 $4.616e-2$ ，真值 $\geq 6.284e-2$ ）；在 CCP 腿的栈上为 34/42。即 strict-64 一直在系统性低报，且低报幅度对两条腿不对称——沿用旧尺子会高估 IPOPT 腿的质量。

2.4 结果图与分析

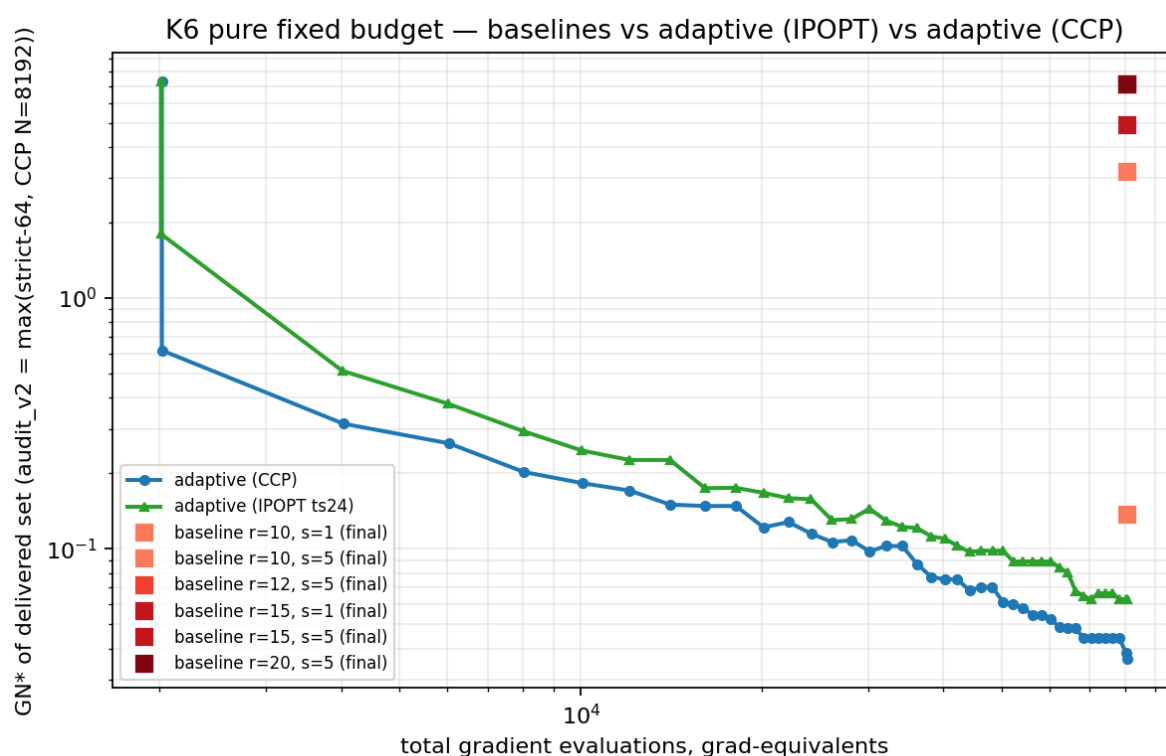


图 2-1 K6：交付集 GN^* (audit_v2) 随梯度当量预算的变化

分析：终点 audit_v2：CCP $3.625e-2$ ，IPOPT $6.284e-2$ （CCP 低 42%）；最好的 baseline（r10_s1） $1.365e-1$ ，其余 baseline ≥ 3.18 （多条卡在顶点平台 7.09 附近——网格分辨率与预算不匹配时扫描策略近乎失效）。值得注意 r10_s1 明显好于所有 s=5 的基线：s=1 每个 segment 就换一次 λ ，扫描更细。两条 adaptive 腿依旧全面占优。

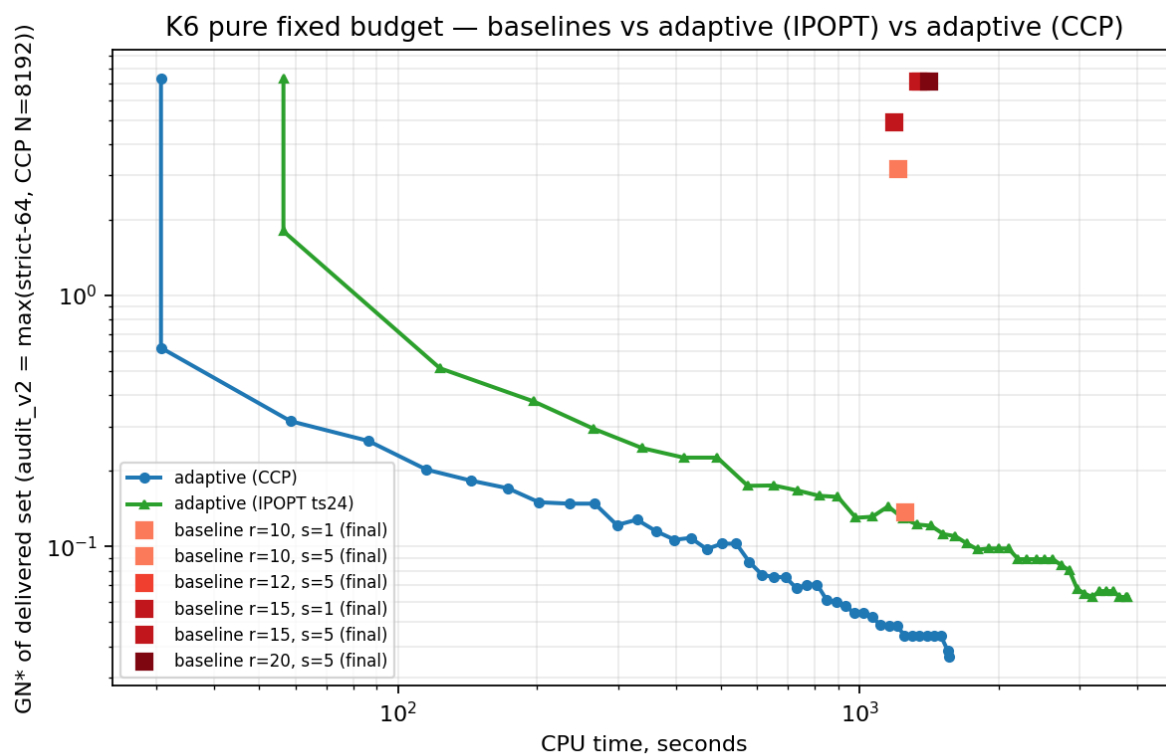


图 2-2 K6 : 交付集 GN* 随 CPU 时间的变化

分析：K=6 下 IPOPT 的 targeting 成本占比升至 70% (2799 s / 3978 s)，CCP 为 31% (484 s / 1575 s)。CPU 轴上 CCP 腿整条曲线大幅左移：整腿快 2.5 倍的同时质量还好 42%。这正是论文的核心论点—— λ 子问题的多起点成本决定了方法在高 K 的可扩展性。

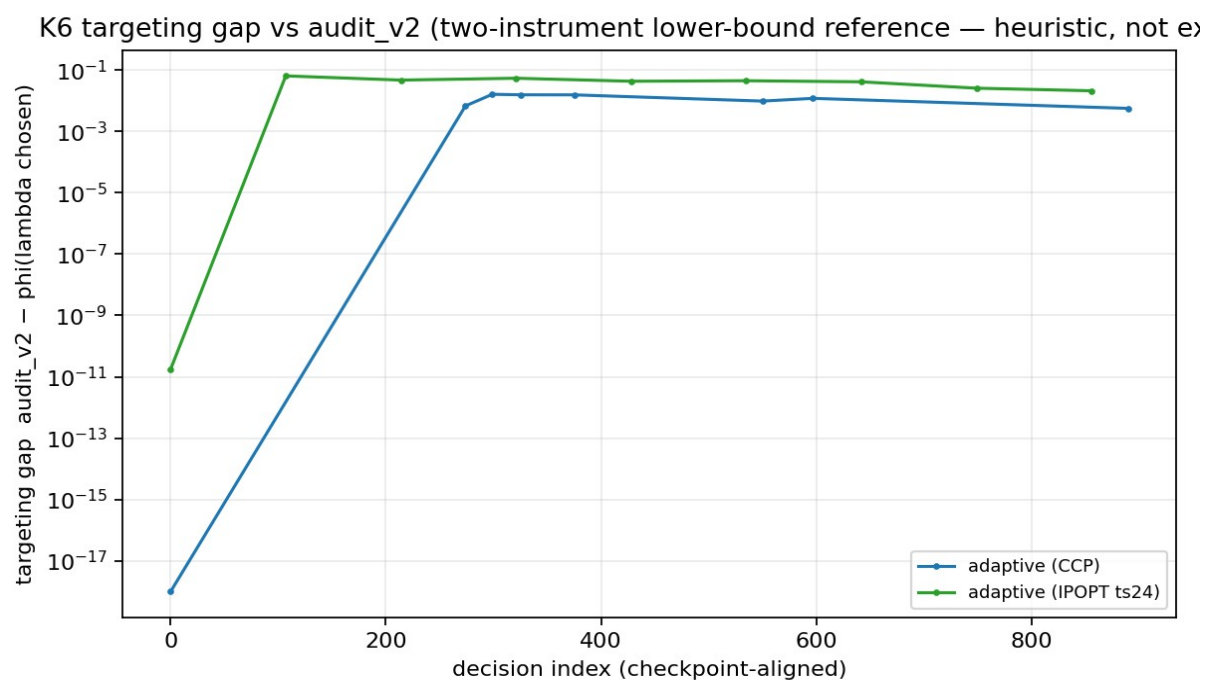


图 2-3 K6 : checkpoint 对齐决策的 targeting gap (参照 = audit_v2)

分析：K=6 没有精确参照，gap 以 audit_v2（两个下界工具的最大值）为参照，逐 checkpoint 对齐决策计算，为启发式度量。需要说明参照包含 CCP 工具，因此 CCP 腿的 gap 有内生优势；稳健的结论方向是：IPOPT 腿的 gap 持续显著为正——它的 targeting 连另一个便宜工具能找到的方向都经常够不到，与 K2 精确尺子下的结论一致。

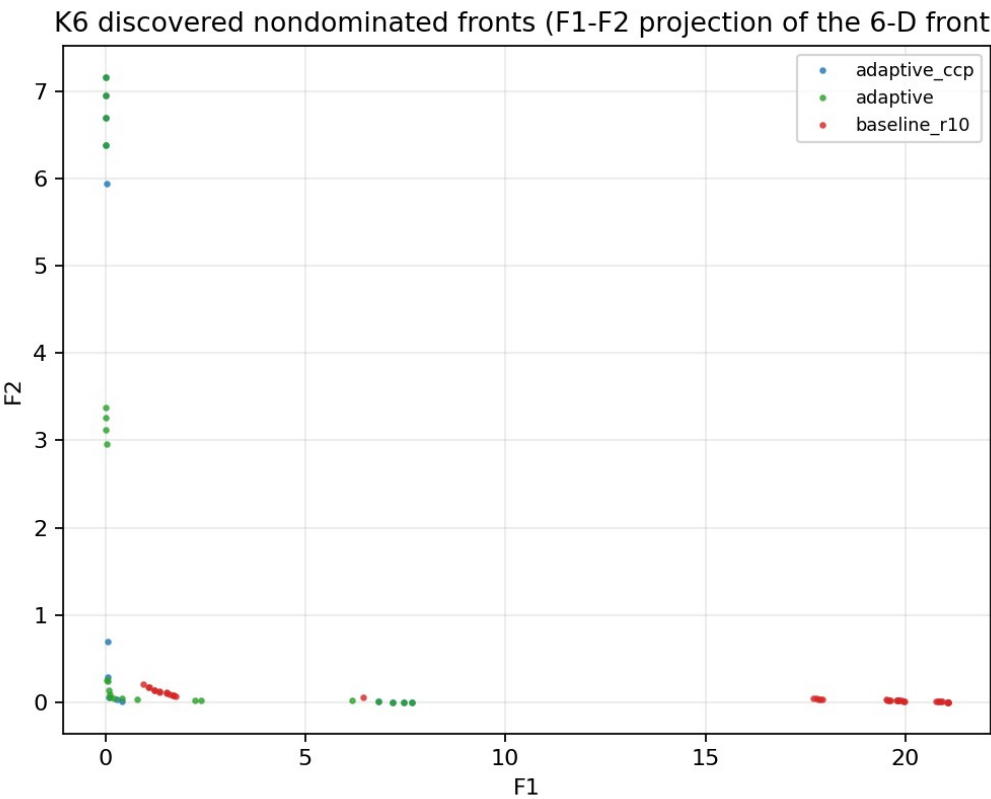


图 2-4 K6：非支配前沿（F1-F2 投影，6 维前沿）

分析：与 K2 相反，K=6 上 adaptive 两腿的前沿指标也占优：IGD 到合并前沿 IPOPT = 2.39（23 点）、CCP = 2.86（14 点）、baseline_r10_s5 = 6.75（39 点）。原因是维度：6 维单纯形上 r=10 网格有 3003 个节点，预算内只能访问约三分之一，扫描式基线的覆盖崩塌；而 adaptive 的“追着最差方向跑”在高维反而给出更有效的前沿采样。CCP 与 IPOPT 前沿质量同级。

备注：第一次跑 baseline_r15_s5 时出现 wall-clock 异常（21252 s，疑似机器休眠；交付集与 audit 逐字节不受影响）。该腿已存档至 baseline_r15_s5_wallclock_artifact/ 并重跑（1338.8 s，正常区间），本报告所有 CPU 数字来自修复后的版本。

2.5 对应的 py 文件与输出

文件	角色
run_pure_budget_K6_without_256_checkpoints.py	协议执行器 + baseline/IPOPT 策略（7 月原文件，未改动）
run_ccp_compare_K6_without_256_checkpoints.py	八条腿同机串行重跑的编排器（新增）

audit_v2_K6_without_256_checkpoints.py	双工具 audit_v2 + gap 数据 (新增 , off-axis)
ccp_lambda_solver.py / algorithm_fast_without_256_checkpoints.py	两个 λ 求解器本体 (同 K2)
plot_ccp_compare_without_256_checkpoints.py	本节全部图
输出目录 : output/ ccp_compare_without_256_checkpoints/K6_B80912/	八条腿 summary/grams + audit_v2.json + 图 + manifest

3. 结论

在两个实例上，相同预算下 CCP 腿交付的点集质量均显著优于 IPOPT 腿（K2 好 12 倍，K6 好 42%），决策成本低 5.4-5.8 倍，整腿 wall time 快 1.5-2.5 倍；K2 的精确逐决策度量表明差距的机制是 IPOPT 结构化多起点对全局最差方向的系统性遗漏（gap 平台 $1e-3 \sim 1e-5$ vs CCP 的 $1e-9$ ）。IPOPT 腿在两个实例上均逐位复现 7 月结果，实验可复现性闭环。统一 audit 在 K6 升级为方法对称的双工具尺子后，结论方向不变且差距扩大。